

Walkthrough: Multi-Role Hiring Automation System

Candidate: Afnan Shoukat **Assignment:** n8n Automation Developer

1. Architecture Overview

The system is designed as a modular, resilient pipeline consisting of two main workflows managed on a single n8n canvas. This separation ensures that high-volume processing and time-sensitive communications are decoupled.

Workflow 1: Application Ingestion & AI Screening

- **Trigger:** Scheduled interval (e.g., hourly).
- **Ingestion:** Fans out to fetch rows from the **Software Engineer (SWE)** and **Business Development Manager (BDM)** Google Form response sheets.
- **Normalization:** A Python-based `Code` node standardizes the divergent schemas (different column names for SWE/BDM) into a unified master schema.
- **Deduplication:** Checks against a **Master Candidates Sheet** using a deterministic `Candidate ID` (SHA-256 hash of `email + role`) to prevent double-processing.
- **Resume Processing:** Downloads the PDF from Google Drive and extracts text content.
- **AI Analysis:** Uses a role-specific LLM prompt (Google Gemini/Groq) to score, classify, and summarize the candidate based on role-relevant rubrics.
- **Persistence:** Updates the Master Sheet with screening results and marks the source sheet rows as `processed`.

Workflow 2: Candidate Communication

- **Trigger:** Scheduled interval (e.g., every 2 hours).
 - **Batching:** Reads all candidates from the Master Sheet where `status = screened` and `contacted = NO`.
 - **Classification Routing:**
 - **Strong:** Creates a **Google Calendar** interview event, sends a personalized invitation email with the meeting link.
 - **Average:** Sends a notification to the **Hiring Manager** for manual review.
 - **Weak:** Sends a professional **rejection email**.
 - **Update:** Marks the candidate as `contacted = YES` in the Master Sheet to prevent duplicate emails.
-

2. Key Architecture Decisions & Rationale

- **Decoupled Pipelines:** Communication is separated from screening. This prevents race conditions, allows for batching emails, and ensures that a failure in the Gmail/Calendar API doesn't crash the core screening logic.
 - **Deterministic Candidate IDs:** By generating IDs from `email + role`, the system is idempotent. If the workflow runs twice on the same data, it update existing records instead of creating duplicates.
 - **Unified Master Store:** Converting SWE/BDM applications into a single format allows the AI and Communication nodes to be "role-agnostic" in logic but "role-aware" in content.
 - **Python for Normalization:** Used Python instead of basic `Set` nodes for data cleaning because it handles messy strings (e.g., placeholder "N/A" values), complex regex for Google Drive URLs, and high-performance deduplication more reliably.
 - **Role-Specific AI Rubrics:** Instead of a generic resume check, the LLM prompt contains distinct evaluation criteria for SWE (technical stack, architecture) vs. BDM (revenue targets, CRM usage), ensuring high-quality assessments.
-

3. Assumptions

- **File Access:** Assumes the service account/API credentials have permission to view files uploaded via Google Forms in the owner's Google Drive.
 - **Resume Format:** Assumes resumes are uploaded as PDFs. The system extracts text directly; scanned images (non-selectable text) without OCR may result in low AI scores due to "missing evidence."
 - **Sheet Structure:** Assumes a `processed` column exists in the source sheets to keep track of what has been ingested.
 - **Role Identifier:** Assumes the `n8n` `Set` nodes correctly tag the source as `SWE` or `BDM`.
-

4. Known Limitations & Improvements

- **PDF OCR:** Currently lacks Optical Character Recognition. If a candidate uploads a "photo" of their resume, the text extraction returns empty. *Improvement: Add an OCR node (like AWS Textract or Tesseract).*
 - **Calendar Logic:** Currently creates an event at a fixed offset (e.g., 3 days from now). *Improvement: Integrate with a tool like Calendly or check the Hiring Manager's availability via API.*
 - **Database Scalability:** Google Sheets is used as the central store for ease of review. *Improvement: For high-volume hiring (10k+ applicants), migrate to Airtable, PostgreSQL, or Pinecone for better performance.*
 - **Human-in-the-Loop:** "Average" candidates rely on a simple email notification. *Improvement: Create a Slack/Discord channel for the hiring team to "Approve" or "Reject" via interactive buttons.*
-

5. Error Handling & Edge Scenarios

- **Missing Data:** The Python code validates every row. If a `name`, `email`, or `resume_url` is missing, the row is skipped and logged as a "skipped row" rather than causing a node crash.
 - **AI Hallucinations:** The `Code1` node parses LLM output and enforces data types. If the AI fails to return valid JSON or returns a score that doesn't match the classification, the code "forces" consistency (e.g., any score ≥ 75 is moved to `strong`).
 - **Graceful Exit:** If no new candidates are found, the `If` node routes to a "No-Op" node that prints a log message and stops execution without error.
 - **Retry Mechanism:** Candidates with `status = screening_error` are automatically retried in the next run, allowing for recovery from temporary API downtime.
-